

Bayesian Statistics Project - Group 6

Dominik Mandić, Fausto Morando, Niccolò Signorelli, Fabio Vicig

12508100, 12508079, 12502991, 12502978

February 4, 2026

1 Task 1: Tournament Ranking

1.1 Problem Definition

The objective of this task is to construct a rating system for a pool of chess engines based on the results of a series of games played between them. We aim to estimate a latent skill parameter (rating) for each engine that predicts the outcome of a game (White Win, Draw, Black Win). The ratings should be formulated with the Elo rating system, centered around a mean of 2000.

1.2 Statistical Methodology

We employ a Bayesian Elo-like model. Let r_i denote the rating of engine i , for $i \in \{1, \dots, 6\}$. For a game between white player w and black player b , we define the effective rating difference Δ_{wb} as:

$$\Delta_{wb} = r_w + \delta - r_b$$

where δ represents the “White advantage”, a parameter that accounts for the intrinsic advantage of moving first.

Following the standard Elo system, the expected score E_{wb} (probability that White wins plus half the probability of a draw) is modeled using the Elo formula:

$$E_{wb} = \frac{1}{1 + 10^{-\Delta_{wb}/400}}$$

However, unlike the binary win/loss formulation, chess outcomes include draws. We model the probability of a draw, p_{draw} , as a function of the absolute skill difference $|\Delta_{wb}|$. It is assumed that players of similar strength are more likely to draw than players with a large skill difference, hence our decision of modeling this quantity in the following way:

$$p_{draw} = p_{base} \cdot \exp\left(-\frac{|\Delta_{wb}|}{\text{draw_scale}}\right)$$

Here, $p_{base} \in [0, 1]$ is the baseline draw probability between equal opponents, and `draw_scale` > 0 is a scaling parameter determining how quickly the draw probability decays as the rating difference increases.

In this setting, the probabilities for the three categorical outcomes

$$y \in \{\text{BlackWin}, \text{Draw}, \text{WhiteWin}\}$$

are then defined as:

$$\begin{aligned} P(y = \text{Draw}) &= p_{draw} \\ P(y = \text{WhiteWin}) &= (1 - p_{draw}) \cdot E_{wb} \\ P(y = \text{BlackWin}) &= (1 - p_{draw}) \cdot (1 - E_{wb}) \end{aligned}$$

where the underlying idea is that the remaining probability mass (after accounting for the draw) is distributed between White and Black proportionally to their Elo expected scores.

1.3 Implementation

The model was implemented in Stan. We specified the following priors:

- Ratings $r_i \sim \mathcal{N}(2000, 200)$, with an additional soft constraint $\text{mean}(r) \sim \mathcal{N}(2000, 10)$ to anchor the global rating mean, following the hint.
- White advantage $\delta \sim \mathcal{N}(35, 15)$, since the advantage is commonly quantified with 35 Elo points (see [1]).
- Draw parameters: $p_{base} \sim \text{Beta}(3, 7)$ and scale `draw_scale` $\sim \mathcal{N}(300, 100)$. Both are weakly informative, with p_{base} having a mean of 0.3 and being right-skewed (favoring lower draw probabilities).

The likelihood is formulated using a categorical distribution, having the outcome probabilities defined above as parameters.

1.4 Conclusions

The posterior estimates for the engine ratings reveal a distinct hierarchy among the competitors (see Figure 1).

- Engine **E** is the clear winner of the tournament. With a mean rating of approximately 2350, its 95% credible interval is entirely separated from the rest of the field, indicating statistically significant superiority.
- Below the leader, we observe two intermediate groups where the skill gap is less pronounced:

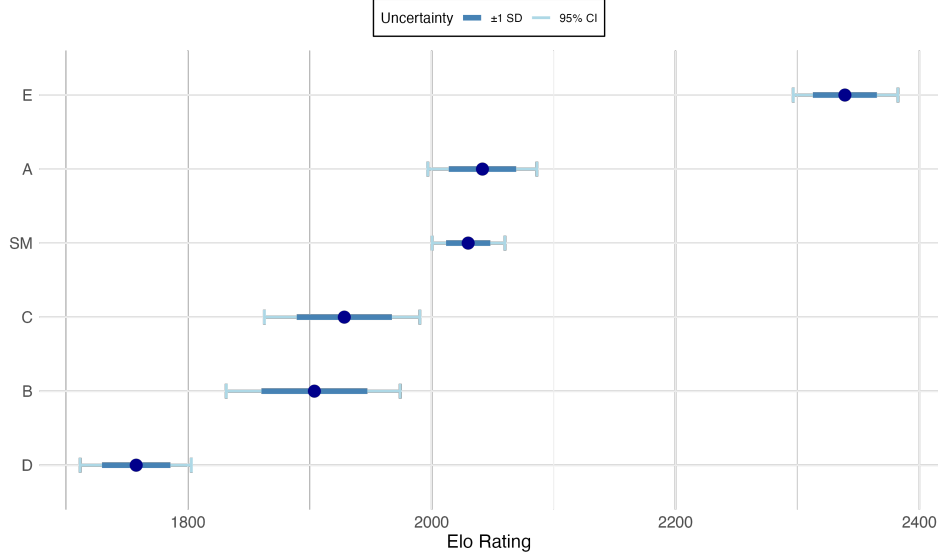


Figure 1: Estimated posterior ratings for all engines. The dots represent the posterior means, while the dark and light blue bars indicate the ± 1 standard deviation and 95% credible intervals, respectively.

- Engines **A** and **SM** form the second tier (ratings $\approx 2000 - 2100$). Their credible intervals overlap significantly, suggesting their strength is comparable, though Engine A has a slightly higher posterior mean (but a wider CI).
- Engines **C** and **B** form the third tier (ratings $\approx 1900 - 1950$). Similar to the group above, their intervals overlap, making it difficult to strictly rank one over the other.
- Engine **D** is clearly the weakest competitor, with a mean rating around 1750. Its entire credible interval lies well below the intervals of all other engines.

Overall, the model successfully recovered the relative skill levels with reasonable precision. The width of the uncertainty intervals is consistent with the data: engines involved in fewer matches (such as B and C) display wider intervals, while the heavily tested engines (SM and E) show tighter bounds. The clear separation between the top, middle, and bottom tiers confirms that the tournament format provided sufficient data to distinguish the engines’ performance levels despite these varying sample sizes.

2 Task 2: Tournament Ranking with Time Control

2.1 Problem Definition

In competitive chess, an engine’s performance is significantly influenced by the available time for calculation. The second task extends the previous ranking model to incorporate time control as a covariate. The goal is to estimate a time-dependent rating $r_i(t)$ for each engine, allowing us to assess how the engines’ ratings depend on thinking time.

Specifically, we are required to produce rating tables analogous to the one in Task 1 for four distinct time controls: 15+0, 60+0, 180+2, and 600+0.

To allow these comparisons, we define the expected game duration $\mathbf{tc}$ for a time control as:

$$\mathbf{tc} = \text{time} + 40 \times \text{increment}$$

This assumes an average game length of 40 moves per side, following the convention used by the Lichess platform [2].

2.2 Statistical Methodology

We modify the static rating r_i from Task 1 to be a linear function of the time control $\mathbf{tc}$. For each engine i , we introduce an intercept α_i and a slope β_i :

$$r_i(t) = \alpha_i + \beta_i \cdot \mathbf{tc}$$

Here, α_i represents the engine’s base strength (in the code it’s represented by the `rating` parameter), and β_i captures the rate of improvement per unit of time increased.

The calculation of the rating difference $\Delta_{wb}(t)$ for a specific game played at time control t becomes:

$$\Delta_{wb}(t) = (\alpha_w + \beta_w \cdot \mathbf{tc}) + \delta - (\alpha_b + \beta_b \cdot \mathbf{tc})$$

where α_w, β_w and α_b, β_b are respectively the parameters for white and black in the i -th game. The rest of the model structure remains identical to the methodology in Task 1, using $\Delta_{wb}(t)$ in place of the static Δ_{wb} .

2.3 Implementation

The Stan implementation from Task 1 is extended by adding the expected game duration data as a new input vector $\mathbf{tc}$, specifying the time control for each game. The effective rating of engine i at time control t is modeled as:

$$r_i(t) = \alpha_i + \beta_i \cdot t$$

where α_i (denoted as `rating` in the code) is the base rating and β_i is the slope.

We specified the following priors:

- Base ratings $\alpha_i \sim \mathcal{N}(2000, 200)$, with the global mean anchored to 2000, exactly as in Task 1.
- Time slopes $\beta_i \sim \mathcal{N}(0, 0.1)$. This prior is relatively informative: we chose a low variance to prevent unrealistic rating drift at higher values of $\mathbf{tc}$ (which in the dataset ranges from 30 to 980).
- White advantage and draw parameters retain the same priors as in Task 1.

The likelihood function remains identical, but now uses the time-adjusted rating difference $\Delta_{wb}(t)$ to compute the expected score and draw probability.

2.4 Conclusions

The posterior estimates for the ratings reveal a distinct hierarchy among the engines, consistent with the ranking observed in Task 1, across all time controls. The results are collected in Table 1.

(a) TC = 15 (Ultra Bullet)				(b) TC = 60 (Bullet)			
Rank	Engine	Rating	95% CI	Rank	Engine	Rating	95% CI
1	E	2325.9	(2270.7, 2382.6)	1	E	2329.5	(2275.9, 2383.6)
2	A	2042.4	(1981.4, 2102.2)	2	A	2042.1	(1983.9, 2098.8)
3	SM	2037.9	(1995.3, 2080.3)	3	SM	2035.4	(1996.0, 2074.6)
4	C	1930.9	(1844.5, 2009.0)	4	C	1930.5	(1846.3, 2006.7)
5	B	1907.2	(1815.1, 1996.3)	5	B	1906.3	(1815.9, 1992.9)
6	D	1754.5	(1691.7, 1813.7)	6	D	1755.1	(1694.6, 1811.1)

(c) TC = 260 (Blitz)				(d) TC = 600 (Rapid)			
Rank	Engine	Rating	95% CI	Rank	Engine	Rating	95% CI
1	E	2345.4	(2288.2, 2402.7)	1	E	2372.5	(2275.9, 2470.4)
2	A	2040.6	(1979.8, 2100.0)	2	A	2038.1	(1934.4, 2140.9)
3	SM	2024.4	(1980.7, 2069.4)	3	SM	2005.6	(1917.5, 2095.4)
4	C	1928.7	(1842.5, 2009.5)	4	C	1925.6	(1805.1, 2045.1)
5	B	1902.2	(1809.8, 1990.8)	5	B	1895.2	(1769.9, 2017.9)
6	D	1757.5	(1694.9, 1816.9)	6	D	1761.7	(1655.5, 1865.8)

Table 1: Posterior rating estimates for different time controls (TC). The tables show the mean rating and the 95% credible interval for each engine.

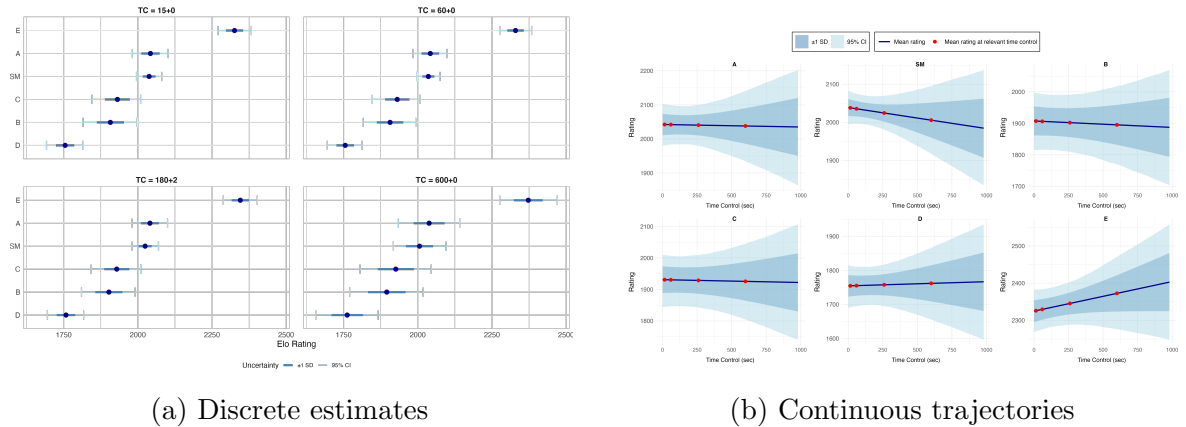


Figure 2: Impact of Time Control. Left (a): Posterior estimates at specific time controls. Right (b): Continuous rating curves showing uncertainty growth as game duration increases.

A key observation from Figure 2 is that the width of the uncertainty intervals increases with the expected game duration. As shown in the continuous curves (Fig.

2b), they widen significantly as the time control increases, reflecting the lower number of observed games for longer matches compared to the faster blitz/bullet games.

3 Task 3

3.1 Problem Definition

For this task the goal is to perform a live prediction for the ongoing tournament. Specifically, we want to estimate the probability that the engine **SM** finishes in either 1st or 2nd place, given the partial results available from the first four rounds. We also have the following decision problem: *we are offered a bet of €100, paid to us if SM finishes in the top two of the tournament. Otherwise, we lose the same amount.*

3.2 Statistical Methodology

To utilize the knowledge acquired about the ratings in the previous tasks and update it with the live tournament information, we use a Bayesian updating scheme. Instead of re-running the whole Stan simulations, our approach is to utilize Sampling Importance Resampling to update our ratings. Finally, we run multiple simulations of the remaining part of the tournament to get the proportion of times in which the bet conditions are satisfied. In each simulation the ratings are drawn from the updated rating posteriors.

3.2.1 Prior and Posterior Update

We treat the posterior samples generated in Task 2 (time-dependent Elo ratings) as our prior distribution for this task. $\theta^{(j)}$ represents the j -th sample vector of ratings from the previous posterior.

We observe a new set of game outcomes D_{new} from the live tournament. To obtain samples from the new posterior $P(\theta \mid D_{new}, D_{old})$, we re-weight the existing samples $\theta^{(j)}$ according to their likelihood of explaining the new data. The weights $w^{(j)}$ are calculated as:

$$w^{(j)} \propto P(D_{new} \mid \theta^{(j)})$$

Once the weights are computed, we resample from the prior set $\{\theta^{(j)}\}$ with replacement, where the probability of selecting sample j is proportional to $w^{(j)}$. This procedure updates the distribution to account for the live tournament results without requiring a whole new simulation run, given that the prior covers the support of the new posterior (which we consider valid in this context, given the broadness of the Task 2 estimates).

3.2.2 Likelihood Function

The likelihood $P(D_{new} \mid \theta^{(j)})$ is computed by aggregating the probabilities of the individual game outcomes in the partial tournament. For a game i with observed outcome y_i and model-predicted expected score $E_i(\theta^{(j)})$, we approximated the likelihood using a Normal density for computational stability:

$$\mathcal{L}_i(\theta^{(j)}) = \mathcal{N}(y_i \mid \mu = E_i(\theta^{(j)}), \sigma = 1)$$

3.3 Implementation

We implemented the prediction utilizing the posterior samples from Task 2 as basis for the analysis. The core mechanism is based on **Sampling Importance Resampling (SIR)** to update our beliefs.

The implementation proceeded as follows:

- **Data Import:** We imported the 8,000 posterior samples generated in Task 2 and the partial results of the live tournament.
- **Likelihood Approximation:** For each of the 8,000 samples, we computed a log-likelihood weight by iterating over all pairings. For a pairing between engines i and j , we calculated the total expected score over 4 games (2 playing White, 2 playing Black):

$$E_{total} = 2 \cdot E_{i,j}(\text{White}) + 2 \cdot E_{i,j}(\text{Black})$$

We then compared this total expected score to the observed aggregate score using a Normal density:

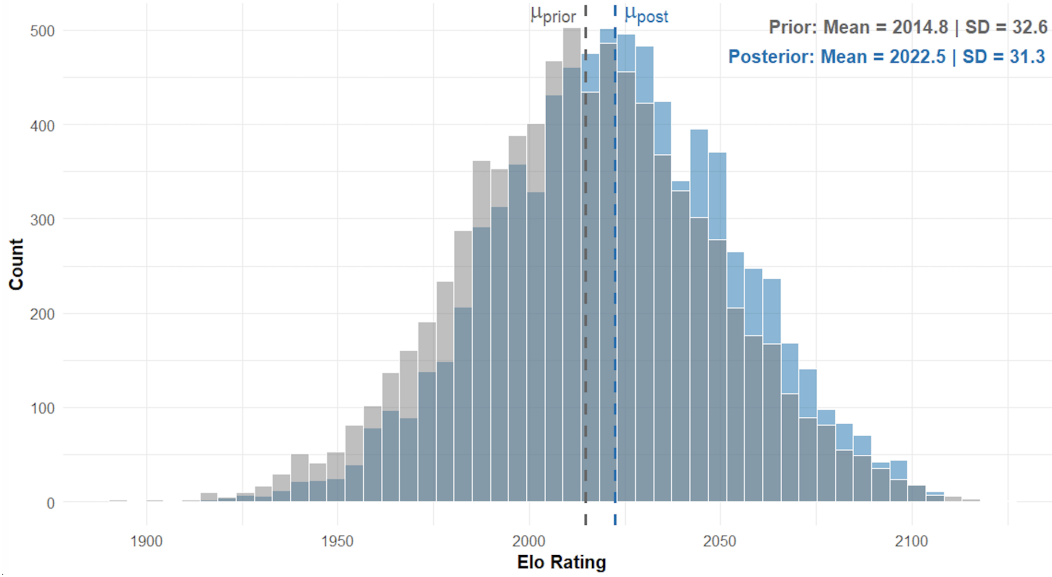
$$\log w^{(k)} = \sum_{\text{pairings}} \log \mathcal{N}(\text{Observed Score}_{i,j} \mid \mu = E_{total}, \sigma = 1.0)$$

The standard deviation of the expected score was manually set to 1, as it looked like a straightforward choice for just four games.

- **Resampling (SIR):** We converted the log-weights to normalized probabilities and resampled 8,000 parameter vectors with replacement. This step filtered the distribution, favoring engine ratings that were most consistent with the observed aggregate scores.
- **Evaluation Metric:** The probability of SM finishing in the top 2 was estimated as the proportion of simulations where its rank was ≤ 2 . This probability was then used to calculate the Expected Utility of the bet.
- **Tournament Simulation:** Using the updated posterior, we simulated the remaining 3 rounds of the tournament 8,000 times. In each simulation iteration:
 1. A unique ratings vector was drawn from the updated posterior.
 2. The outcome of every unplayed game was simulated using the categorical probabilities derived from our ELO model (see Task 1).
 3. Final scores were saved to produce a complete ranking for each simulation.

Figure 3 shows the effect of the updates for SchachMaus’s rating.

Figure 3: Comparison of SchachMaus rating samples before and after integrating live results.



3.4 Conclusion

According to the simulation results, the expected utility was:

$$\mathbb{E}[U] = 100 \cdot P(\text{rank} \leq 2) + (-100) \cdot P(\text{rank} > 2) \approx 25 \text{ EUR}$$

Since $\mathbb{E}[U] > 0$, and being us not adverse to gambling and risk when the odds are in our favour, we would decide to accept the bet!

4 Task 4: Sequential Testing

4.1 Problem Definition

The objective of this task was to develop a method for assessing whether a modification to a chess engine yields an improvement over the base version with default parameters. A naive approach would be to play a fixed number of games (e.g., 100) and calculate the win rate. However, this approach is inefficient. To solve it, we implemented a Sequential Probability Ratio Test (SPRT) [3] based on the parameters derived from our previous Bayesian model.

4.2 Statistical Methodology

Instead of a fixed sample size, SPRT evaluates the data as it arrives (game by game). The core decision to use this method rests on defining the “improvement” not as a raw win-rate percentage, but as a latent Elo difference. Since this value is unknown, we must infer it through sequential sampling to determine the difference between models.

For testing, we defined two point hypotheses following the standard SPRT formulation:

- H_0 : $\Delta\text{Elo} = 0$ (No improvement).
- H_1 : $\Delta\text{Elo} = E_0$ (Improvement by target margin, e.g., 10 Elo).

Note: Although the goal is to verify if the improvement is *at least* E_0 , mathematically SPRT compares the likelihood of the data under these two specific points. Empirically, if the true strength difference is greater than E_0 , the test will converge to H_1 even faster.

4.3 Implementation

4.3.1 Integration of Bayesian Priors

We utilized the posterior means extracted from our fitted Stan model (from Task 2) to calculate the exact probabilities for every specific game using the engine-specific parameters.

Time Control Sensitivity: We utilized the β parameter from our model to adjust the expected performance based on the specific time control (tc) of the current game. Specifically, we calculate the effective rating difference for each game as:

$$\Delta_{\text{eff}} = \Delta_{\text{Elo}} + (\beta_{\text{new}} - \beta_{\text{base}}) \cdot tc$$

This ensures the test is valid regardless of whether the game is Bullet or Rapid.

Draw Probability: Maintaining the structure of Task 2’s model, we incorporated the modeled p_{draw} and draw_scale . Crucially, we employed an exponential decay model:

$$P(\text{draw}) \sim \exp\left(-\frac{|\Delta_{\text{eff}}|}{\text{scale}}\right)$$

based on the assumption that mismatched opponents draw less frequently. This ensures that the likelihood of a draw is accounted for mathematically, rather than treating draws simply as “half a win”.

First-Move Advantage: The white_advantage parameter was explicitly included in the expected score calculation, preventing color imbalances from skewing the test results.

4.3.2 Simulation & Validation

We simulated the testing process using the historical game data. After defining the error rates for type 1 and type 2 errors α and β , the test consisted in calculating the Log-Likelihood Ratio (LLR) after every game:

- If $\text{LLR} \geq B$ (derived from β error), we accept H_1 (New engine is better).

- If $\text{LLR} \leq A$ (derived from α error), we accept H_0 (No improvement).

To determine the stopping thresholds, we employed Wald’s approximation. Instead of arbitrary cutoff points, the bounds A and B were derived directly from our chosen error rates α and β using logarithmic scales:

$$A = \ln \left(\frac{\beta}{1 - \alpha} \right) \quad \text{and} \quad B = \ln \left(\frac{1 - \beta}{\alpha} \right)$$

We selected $\alpha = \beta = 0.20$ for this retrospective simulation to allow the test to reach conclusions despite the limited number of historical games available in the dataset.

Furthermore, while the standard Elo formula provides only an expected score, our SPRT requires exact probabilities for all three outcomes (Win, Loss, Draw). We modeled this by decomposing the outcome probabilities: we first modeled the probability of a draw (p_{draw}) using our Bayesian parameters, and then distributed the remaining probability mass to Win/Loss outcomes according to the logistic Elo curve.

Validation Result: The simulation demonstrated that for pairs where the Elo difference (calculated in Task 1) was large, the SPRT stopped very early, validating the efficiency of the method. For pairs with similar ratings, the test naturally required more samples to distinguish signal from noise. In cases where the test did not conclude within the available historical games, the result remained “Undecided”.

4.4 Conclusion

The result of testing with $E_0 = 10$ Elo, $\alpha = 0.20$, $\beta = 0.20$ are shown in Table 2:

Model	A	B	C	D	E	SM
A		?	?	?	-	?
B	?		?	?	?	?
C	?	?		?	?	?
D	?	?	?		-	-
E	+	?	?	+		+
SM	?	?	?	+	-	

Table 2: SPRT results (+: engine in row better, -: engine in row worse, ?: undecided)

5 Task 5: Tuning using Sequential Testing

5.1 Problem Definition

The objective of this task was to establish a performance baseline for the **SchachMaus** engine. As outlined in the assignment, the goal was not to find a global optimum immediately, but to identify a reasonable parameter set via manual tuning that outperforms the default configuration, thereby creating a solid foundation for the subsequent automated Bayesian optimization.

5.2 Statistical Methodology

We adopted an incremental tuning approach utilizing Sequential Testing. Starting from the engine’s default behavior, we hypothesized parameter adjustments based on theoretical heuristics and tested them sequentially.

- **Baseline:** The default parameter set of `SchachMaus` ($NMP_minDepth=3$, $RFP_intercept=-30$, $LMR_intercept=0$).
- **Candidates:** We evaluated iterative improvements:
 1. **Configuration A (Aggressive Pruning):** We adjusted $NMP_minDepth$ to 4 and RFP_slope to 150. This aimed to test if the engine benefits from pruning search branches earlier to achieve greater depth.
 2. **Configuration B (Validation):** A secondary configuration with relaxed pruning constraints to verify the sensitivity of our SPRT setup.
- **Testing Environment:** To ensure the independence of game results required by the statistical test, we utilized the `8moves_v3.epd` opening book, forcing the engines into diverse starting positions.

5.3 Implementation

To assess the candidates efficiently, we employed the Sequential Probability Ratio Test (SPRT) implemented in Task 4.

- **Hypotheses:** We defined the null hypothesis as an Elo improvement of $Elo_{diff} \leq 0$ against the alternative of $Elo_{diff} \geq 10$ (with error rates $\alpha = \beta = 0.05$).
- **Execution:** Matches were played with a time control of 1 + 0.1 (Bullet).

5.4 Conclusion

The testing procedure revealed that the more aggressive pruning configuration (Configuration A) yielded a statistically significant improvement over the baseline. Consequently, this set was selected as the new default for the automated Bayesian optimization tasks.

It is worth noting that despite using SPRT, the testing process was computationally intensive. Since the actual strength difference between parameter sets was often small (close to the decision boundaries), the test required a large number of games to converge, consuming significant runtime. Finally, the fitted parameter vectors were used to initialize the Gaussian Processes in the following tasks.

6 Task 6: Automated Parameter Tuning

6.1 Problem Definition

In the previous task, it was necessary to manually tune the parameters of the **SchachMaus** engine by letting it play against itself with different settings. To evaluate which version performed best, a sequential testing approach was implemented. This approach allowed us to gather information on the problem, which will be used in the following task. The goal now is to optimize one specific parameter, **LMR_slope** (the Late Move Pruning slope) for an arbitrary but fixed time control, using Bayesian optimization. In other words, the goal is to find the value of x that maximizes an unknown function $f(x)$, which cannot be directly measured and is computationally expensive to evaluate. In this case, the parameter x corresponds to the **LMR_slope**, and the function $f(x)$ represents the posterior mean of the rating, estimated through the Bayesian model described in task 1.

6.2 Statistical Methodology

Assuming that a reasonable range for the objective parameter is between 0 and 2, and fixing the other parameters to specific values, the first step is to randomly sample four values for the **LMR_slope**. For each value, we create a candidate engine, and a round-robin tournament is held, where 20 is the number of games.

Given the observed evaluations y_i and their standard deviations s_i , it is not possible to evaluate exactly the unknown function $f(x_i)$. Therefore, an additive noise term ϵ_i is introduced for $i = 1, \dots, 4$. Such unknown function is approximated using a surrogate model, and a common choice is a Gaussian Process

$$f \sim \mathcal{GP}(\mu_D, k_D),$$

where μ_D and k_D are the posterior mean and kernel conditioned on the data $D = (x, y)$. Assuming a Gaussian kernel and additive Gaussian noise with variance proportional to the estimated standard deviations, the following model is adopted

$$y_i = f(x_i) + \epsilon_i \quad \epsilon_i \sim \mathcal{N}(0, \sigma_i^2),$$

where $\sigma_i^2 = \frac{s_i^2}{2}$. Based on the posterior distribution of the model, we compute its predictions to select the next candidate to be sampled. Indeed, the next query point x' is determined by maximizing the acquisition function $\alpha(\tilde{x} \mid D)$ derived from the Gaussian Process. Two acquisition functions are considered:

- **Expected Improvement**, which provides a good balance between exploration and exploitation;
- **Probability of Improvement**, which favors exploitation.

They both compare the posterior predictions with the best value observed so far, i.e., $\max y$ in D .

6.3 Implementation

The results from the tournament are slightly adjusted and then used as observations for the Bayesian model.

	engine	mean_rating	sd_rating	LMR_slope
1	A	2086.268	29.27428	0.2
2	B	2016.076	27.36509	0.9
3	C	1967.658	25.66844	1.3
4	D	1930.368	23.74088	1.6

Table 3: Posterior means and posterior standard deviations of the ratings, with their random parameters for LMR_slope.

In order to fit a Gaussian Process to the data, we use the function `km()` from the R package `DiceKriging`, specifying the kernel type and the noise variance.

```
gp_model <- km(formula=~1, design=x, response=y,
               covtype="gauss", noise.var=(df_results$sd_rating/2)^2)
```

Using a grid of values in the interval $[0, 2]$, we compute the posterior mean and standard deviation. As mentioned above, one of these values is then selected as the next candidate point to be sampled.

```
z <- (pred$mean - max(df_results$mean_rating)) / pred$sd
PI <- pnorm(z)
EI <- (pred$mean - max(df_results$mean_rating)) * pnorm(z) + pred$sd * dnorm(z)
```

By plotting these measures on the grid, the value $x' = 0$ is selected as the next query point.

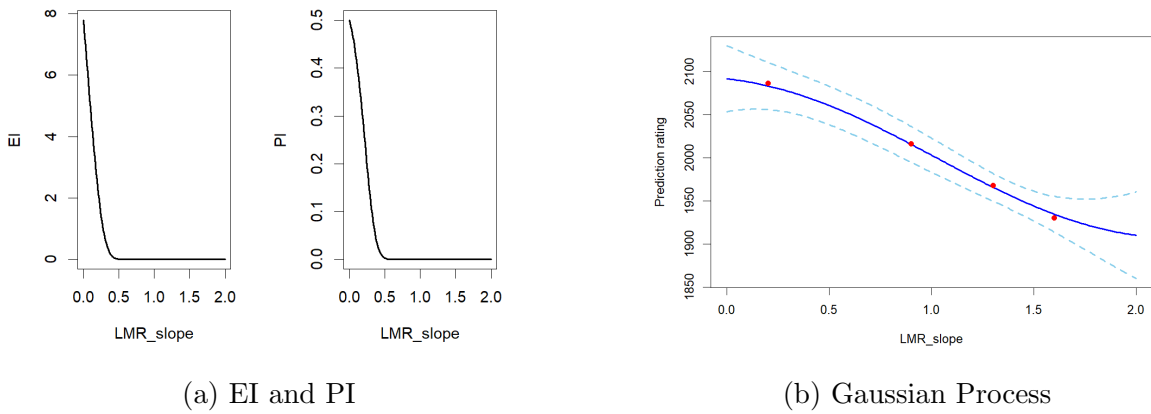


Figure 4: Step 1. Left (a): Expected Improvement and Probability Improvement. Right (b): Posterior predictions, 95% confidence interval and observed data.

After collecting a new sample, an additional tournament is performed. In this stage, only the matches involving the new candidate E, which is set with `LMR_slope` = 0, are considered. The resulting data are used as new inputs for the Bayesian model, and only the posterior mean and standard deviation of the new candidate are retained. Such values are combined with the previous observations to obtain an updated dataset. We repeat this process 3 times, until a reasonable and stable result is obtained.

	engine	mean_rating	sd_rating	LMR_slope
1	A	2086.268	29.27428	0.2
2	B	2016.076	27.36509	0.9
3	C	1967.658	25.66844	1.3
4	D	1930.368	23.74088	1.6
5	E	2053.813	27.69631	0
6	F	2098.182	29.15255	0.3015
7	G	2036.719	24.34163	0.3618

Table 4: Posterior means and posterior standard deviations of the ratings, with their random parameters for `LMR_slope`.

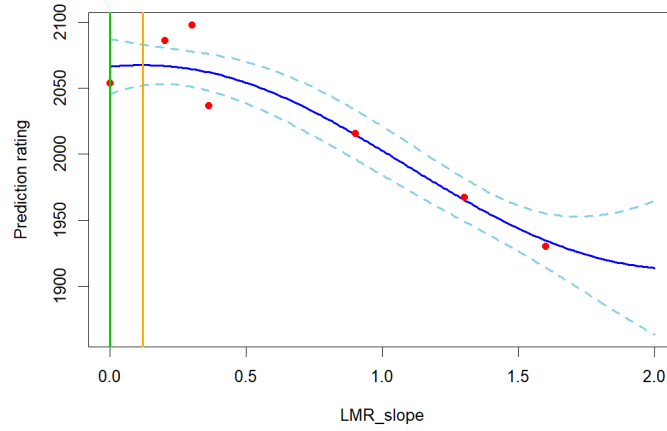


Figure 5: Step 3. Posterior predictions, 95% confidence interval, observed data.

6.4 Conclusion

According to the Expected Improvement criterion, the next query point would again be 0, whereas the Probability of Improvement suggests selecting `LMR_slope` = 0.120603. However, it is decided to stop the Bayesian optimization and eventually, consider the value 0.3015075 as the optimal parameter value.

7 Task 7: Automated Parameter Tuning - 2-D

7.1 Problem Definition

Using the previous results, the focus is now on simultaneously optimizing two correlated parameters: `RFP_intercept` and `RFP_slope`. Based on expert knowledge, it is assumed that `RFP_intercept` lies in the range $[-100, 300]$, while `RFP_slope` lies in $[0, 500]$, with the additional constraint that their sum must be positive.

7.2 Statistical Methodology

Such task is similar to the previous one, with the main difference being the use of a new kernel for the Gaussian Process. Instead of a kernel with two parameters, the model now depends on the variance σ^2 and two length scales l_1 and l_2 :

$$k(x, y) = \sigma^2 \exp\left(-\frac{(x_1 - y_1)^2}{2l_1^2}\right) \exp\left(-\frac{(x_2 - y_2)^2}{2l_2^2}\right).$$

7.3 Implementation

As in Task 6, the first step consists of randomly sampling a set of candidate values. These candidates are then used to create engines that compete against each other in a round-robin tournament. The sampling is performed from a uniform distribution $U(-100, 300)$ for the first parameter and from a uniform distribution $U(0, 500)$ for the second one. Besides, the previously obtained optimal value for `LMR_slope` is used. The tournament results, together with the posterior means and standard deviations obtained from the Bayesian model, are reported.

	engine	mean_rating	sd_rating	RFP_intercept	RFP_slope
1	A	1994.924	22.09112	15.03101	394.15257
2	B	1989.855	29.03309	63.59077	441.50870
3	C	2033.920	29.46759	276.18691	22.77825
4	D	1980.808	28.60272	111.24220	446.20952

Table 5: Posterior means and posterior standard deviations, with their random parameters for `RFP_intercept` and `RFP_slope`.

At this stage, the Gaussian Process is fitted to the data, and the estimated parameters are presented. It is worth noting that the two theta parameters do not directly correspond to the length scales, since the R function `km()` adopts a different parametrization.

```
gp_model <- km(formula=~1, design=x, response=y,  
               covtype="gauss", noise.var=(df_results$sd_rating/2)^2)  
> gp_model@covariance
```

```
Covar. type : gauss
```

Covar. coeff.:

	Estimate
theta(RFP_intercept)	522.3118
theta(RFP_slope)	147.6853

Variance estimate: 317.1093

According to the Expected Improvement criterion, the next query point is $\text{RFP_intercept} = 300$ and $\text{RFP_slope} = 0$, whereas the Probability of Improvement suggests $\text{RFP_intercept} = 277.8894$ and $\text{RFP_slope} = 17.58794$.

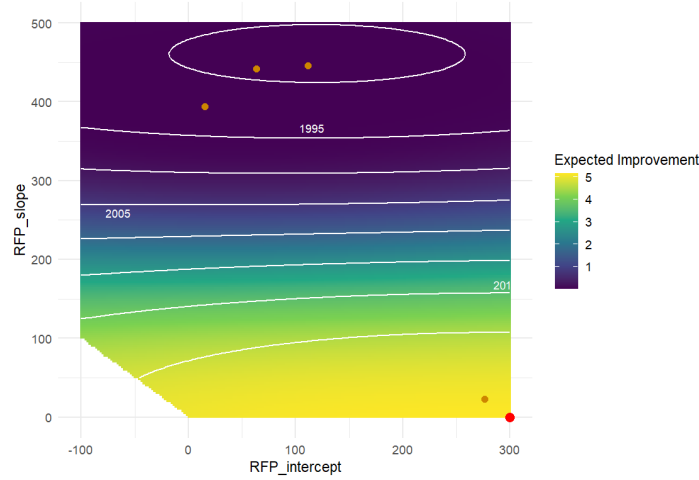


Figure 6: Step 1. Contour plot for the fitted model, the observed evaluations (orange), and the next query point (red).

The process is repeated until the acquisition functions seem to indicate the same next query points. The final step leads to the following results and the following fitted model.

	engine	mean_rating	sd_rating	RFP_intercept	RFP_slope
1	A	1994.924	22.09112	15.03101	394.15257
2	B	1989.855	29.03309	63.59077	441.50870
3	C	2033.920	29.46759	276.18691	22.77825
4	D	1980.808	28.60272	111.24220	446.20952
5	E	2035.892	24.47526	300	0
6	F	1873.652	30.01756	0.50251	0
7	G	2010.385	22.59287	300	145.72864
8	H	2010.929	22.20517	-100	500

Table 6: Posterior means and posterior standard deviations, with their random parameters for RFP_intercept and RFP_slope .

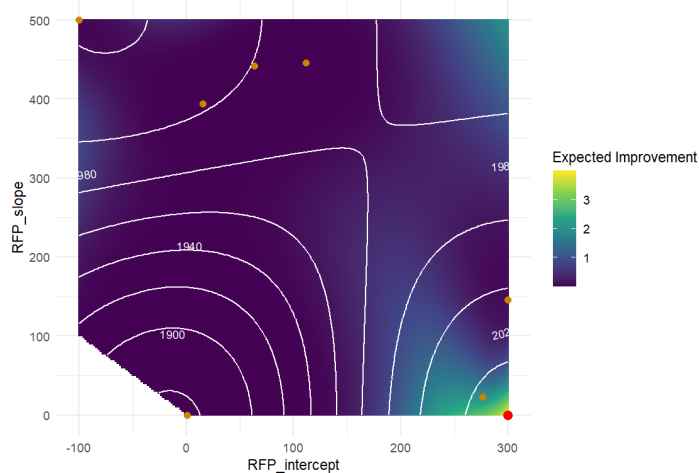


Figure 7: Step 4. Contour plot for the fitted model, the observed evaluations (orange), and the next query point (red).

7.4 Conclusion

According to both methods, the next query point would be again $\text{RFP_intercept} = 300$ and $\text{RFP_slope} = 0$. Therefore it is decided to stop the Bayesian optimization and such optimal values are selected as the best decision for the considered parameters.

8 Task 8: MCMC implementation

8.1 Problem Definition

This task involves running a comprehensive tournament to compare four distinct versions of our chess engine:

1. **Base**: The default, unoptimized engine.
2. **Manual**: A manually tuned version using parameters from Task 5.
3. **LMR_Opt**: An optimized version incorporating the best LMR parameter found in Task 6.
4. **RFP_Final**: The final optimized engine, obtained in Task 7 through the 2D optimization of the RFP parameters.

The objective is to estimate the Elo rating of these four versions to verify whether the sequential tuning steps resulted in actual performance improvements. Unlike previous tasks where we relied on Stan for inference, it is now required to implement a custom Markov Chain Monte Carlo (MCMC) sampler in R to estimate the posterior distribution of the ratings.

8.2 Statistical Methodology

We employ the same Bayesian Elo model defined in Task 2 for the estimates of the engines' ratings. Since the tournament was conducted at a single fixed time control (10+0.1, in seconds), the time-dependent term $\beta \cdot t$ from Task 2 is theoretically redundant. However, to maintain consistency with the full model structure defined previously, we retained the linear term $r_i(t) = \alpha_i + \beta_i \cdot t$ in the sampler, while treating the time control `tc` as a constant ($t = 14\text{s}$).

The data D was generated by the simulation of a 50 rounds Round-Robin tournament. To ensure fairness and variability, each matchup sampled a random opening position from the opening book `8moves_v3.epd`, and players alternated colors for each opening. The time control was set to 10 seconds base time plus 0.1 seconds increment as required.

8.3 Implementation

We implemented a custom **Random Walk Metropolis-Hastings (MH)** sampler for our analysis. This algorithm was chosen because it offers an optimal balance between implementation simplicity and statistical validity. Unlike Gibbs sampling, MH does not require deriving complex conditional distributions; unlike Hamiltonian Monte Carlo, it avoids gradient calculations. Given the assignment's focus on understanding Bayesian mechanics, the MH algorithm provides a robust, gradient-free solution. Furthermore, by employing a symmetric Gaussian Random Walk proposal, the Hastings correction in the acceptance ratio cancels out, simplifying the implementation to the standard Metropolis algorithm.

The MCMC sampler was structured as follows:

- **Model Structure:** Consistent with Task 2, we retained the full linear model $r_i(t) = \alpha_i + \beta_i \cdot t$. Although the time control was constant ($t = 14\text{s}$), keeping the β term maintains model generality.
- **Priors:** We used the same informative priors as in previous tasks: $r_i \sim \mathcal{N}(2000, 200)$, $\delta \sim \mathcal{N}(35, 15)$, $p_{base} \sim \text{Beta}(3, 7)$, and `draw_scale` $\sim \mathcal{N}(300, 100)$.
- **Proposal Distribution:** We employed a random-walk Metropolis proposal. For each parameter ϕ , at each iteration k a new value ϕ^{k+1} was proposed via $\phi^{k+1} \sim \mathcal{N}(\phi^k, \sigma_{prop}^2)$. The proposal standard deviations σ_{prop} were manually tuned (e.g., $\sigma_{rating} = 30$, $\sigma_{beta} = 0.02$) based on the magnitude of each parameter, allowing for decent acceptance rates.
- **Boundary Handling:** For constrained parameters like `draw_scale` > 0 and $p_{base} \in [0, 1]$, we used reflecting boundaries in the proposal step to ensure validity.
- **Acceptance:** The acceptance probability for each update step was computed as $\min(1, \exp(\log P(\phi^{k+1}|D) - \log P(\phi^k|D)))$, where $P(\phi|D)$ is the unnormalized posterior density and D is the generated data.

- **Algorithm Settings:** The sampler was run for 100,000 iterations with a burn-in of 10,000 on two chains. Samples were thinned by a factor of 50 to reduce autocorrelation.

8.4 Conclusions

The convergence of the MCMC sampler was assessed using the Gelman-Rubin diagnostic $\hat{R}$ across two independent chains. All parameters, including the ratings for each engine, yielded $\hat{R}$ values effectively equal to 1.0 (max $\hat{R} \approx 1.002$), indicating that the chains have successfully converged to the stationary posterior distribution.

The tournament results confirm a progressive improvement in engine strength corresponding to each tuning step, as visualized in Figure 8.

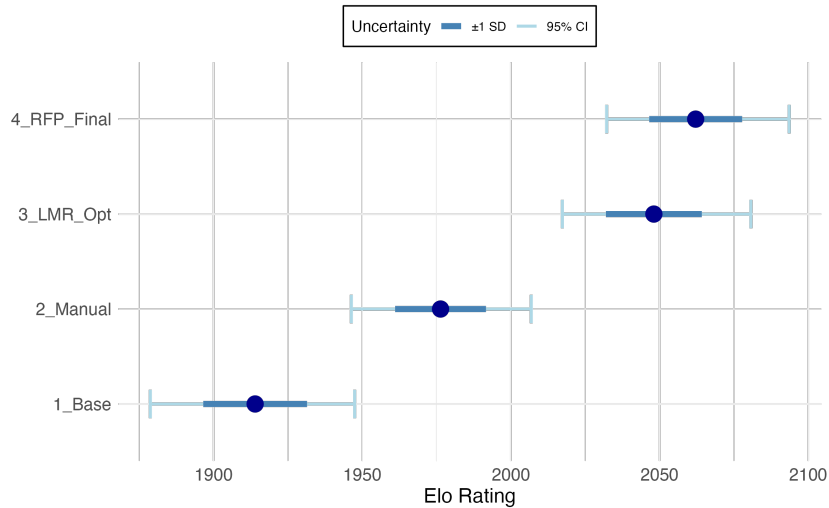


Figure 8: Posterior estimates of Elo ratings for the four engine versions. The dots represent the posterior means, while the dark and light blue bars indicate the ± 1 standard deviation and 95% credible intervals, respectively.

The final optimized engine **4_RFP** achieved the highest mean rating of 2062 Elo, demonstrating a substantial gain of performance compared to the previous versions, even though its Credibility Interval overlaps with **3_LMR**'s.

To quantify these improvements, Table 7 presents the pairwise probability of superiority. It confirms that the first three versions are strictly separated ($P \approx 1.0$), while the final step shows a probabilistic improvement ($P = 0.78$).

Overall, the tuning process was successful, with each stage contributing positively to the engine's playing strength.

9 Task 9: Conclusion

The project can be subdivided into two main parts:

Table 7: Probability of Superiority $P(\text{Row} > \text{Column})$. Values near 1.0 indicate the Row engine is strictly stronger.

Engine	1_Base	2_Manual	3_LMR	4_RFP
1_Base	–			
2_Manual	1.00	–		
3_LMR	1.00	1.00	–	
4_RFP	1.00	1.00	0.78	–

1. **Tournament Analysis:** We developed a way of estimating the strength of engines based on the results they obtain.
2. **Engine Optimization:** We tuned the parameters of the SchachMaus engine to maximize its performance, and ran tournament simulations to confirm the improvements via a custom MCMC implementation.

The results of our work are summarized in Table 8.

Table 8: Final Optimized Parameters (Engine 4_RFP)

Parameter	Optimal Value
NMP intercept	4
LMR intercept	1
LMR slope	0.3015
RFP intercept	300
RFP slope	0

Despite all the work that has been done, there are still a couple of points that could be addressed to produce even better and more realistic estimates of the ratings, and for improving the performance of SchachMaus:

- **Linearity of time control effect:** It’s an assumption that simplifies calculations, but especially for longer time controls it most likely overestimates variations in ratings, especially for engines with significant differences between short and long time controls. By keeping it linear, the difference in rating for between a 1+0 bullet game and a 11+0 rapid game would be the same as the difference between the strength of two classical time controls, e.g. 110’+0 and 120’+0, which clearly can’t be compared. There is a reason why FIDE keeps Blitz, Rapid and Standard ratings separate [4].
- **Greedy optimization of dependent parameters:** The sequential optimization approach assumes that parameters optimized in different stages do not strongly interact. If a parameter tuned in Task 6 has a strong correlation with a parameter tuned in Task 7, fixing the first one early prevents the model from exploring the joint domain. Ideally, all parameters should be optimized simultaneously (joint optimization), but this is of course computationally infeasible.

- **Assumption of smoothness in GP models:** We used Gaussian Processes with stationary kernels to model engine strength. This assumes that the relationship between parameters and Elo rating is smooth and continuous. However, software parameters could exhibit discontinuities where a small change leads to a sudden drop in performance (e.g., due to bugs or instability) rather than a smooth degradation. Our model may fail to capture these non-smooth regimes accurately.
- **Overfitting to the pool:** We optimized our engine by playing primarily against previous versions of itself or a small pool of fixed opponents. This risks overfitting the parameters to exploit specific weaknesses of this small pool rather than increasing general chess strength. An improvement in rating within this closed ecosystem does not guarantee an improvement against the broader field of external engines.
- **Increase number of games and game length for tournament simulation:** Due to our limited resources, tournament simulations have been run only with games of short duration. With more time and computing power available, we would be able to produce more accurate results by tightening credibility intervals: more games would be beneficial for that, and longer games would allow us to diminish the need to extrapolate ratings, especially given the linear relation for time control discussed above.
- **Using Tablebase Endgames:** Implementing Tablebase endgames into our engine would mean that it would “know” whether each endgame with 7 or less pieces on the board is a theoretical win, draw or loss, simplifying calculations in endgames.
- **Games against real players:** Games against real players could potentially confirm the ideas we have about the ratings, although results would still need to be taken with a grain of salt, due to psychological aspects of playing against an engine, different playing styles, and more.
- **Refined estimate of draw probability:** Despite our method for calculating the probability of drawing being already slightly elaborate, it could be fine-tuned given other real-world data about draws between engines. Importantly, it should also be adapted to the time control, as shorter time controls tend to make for games with a higher proportion of decisive results.

Maybe SchachMaus will never be as good as Stockfish [5], but the road that leads there passes through these and many more refinements.

References

- [1] Jeff Sonas. The sonas rating system and statistical analysis. Chessmetrics, 2005. Retrieved from <http://www.chessmetrics.com>.
- [2] Lichess.org. Frequently asked questions: How are bullet, blitz and other time controls decided?
- [3] Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.
- [4] FIDE. Fide ratings database. <https://ratings.fide.com/>.
- [5] The Stockfish Developers. Stockfish – open source chess engine. <https://stockfishchess.org/>, 2026. Accessed: 2026-02-03.